FIELD-BASED CAPSTONE SCENARIO Tashkent House Price Predictor Real Estate | Technical Project Brief Track: Field-Based Scenario Client: Real estate stakeholders in Tashkent (buyers, sellers, agents, analysts) Purpose: Design and implement a supervised machine learning solution that predicts residential property prices in Tashkent --- 1. Client Background -------------------- Tashkent is undergoing rapid urbanization, producing a dynamic and often opaque real estate market. Buyers, sellers, and l 2. Business Problem ------------------- There is no standardized, data-driven method to estimate fair market prices for residential properties in Tashkent. This inform - Buyers possibly overpaying due to lack of comparable objective estimates. - Sellers underpricing or being unable to justify their asking price. - Agents and analysts relying on intuition rather than reproducible analysis. The business need is a transparent, reproducible pricing tool that estimates property values from observable characteristics 3. Requested Solution --------------------- Develop a supervised regression solution to predict residential property prices in Tashkent using the publicly available "Hous - A reproducible end-to-end Colab notebook demonstrating data processing, model training, evaluation, and inference. - A saved model and preprocessing pipeline for inference (model.pkl). - Documentation (dataset license/notes, evaluation results, limitations). - A demo inference helper (e.g., predict() function) for quick price estimates on new properties. 4. Available Information ------------------------ Dataset: "House Prices in Tashkent" (Kaggle) â Apache 2.0 license. Key fields available and used: - Location: District or neighborhood (categorical) - Total_Area_sqm: Total area in square meters (numeric) - Rooms: Number of rooms (numeric) - Floor: Floor number of the unit (numeric) - Total_Flats (Total number of floors in the building) (numeric) - Price: Target variable (numeric, UZS) Notes: - Data is publicly available and contains anonymized property listings. - No personally identifiable information (PII) is present. - Additional metadata (if present) must be reviewed for leakage risk. 5. Data & Problem Discovery â Complete Before Modeling -- Decision / Question | Student Response:--- | :--- Selected dataset and source | Dataset: "House Prices in Tashkent" (Kaggle). Source: Kaggle dataset page. License: Apache What does one record / sample represent? | One record = single residential property listing in Tashkent with features and lis Proposed target or ML objective | Supervised regression. Target: Price (UZS). Objective: Predict numeric price from property Key information available at prediction / inference time | Location (district), Total_Area_sqm, Rooms, Floor, Total_Flats â al Main data quality issues | Check for: missing values, inconsistent district names/spellings, outliers (area, price), invalid floor Potential leakage risks | Avoid features derived from the target (e.g., price per sqm). If any target-derived encodings or post- Privacy / fairness / licensing concerns | Dataset is under Apache 2.0 and anonymized. Evaluate fairness by inspecting per-d 6. Technical Proposal â Complete Before Full Implementation --- Decision / Question | Student Response:--- | :--- ML problem formulation | Supervised regression: learn mapping f(features) â Price. Proposed baseline | Linear Regression (Ordinary Least Squares). Use Ridge variant as a robust baseline if regularization he Main modeling approach(es) to investigate | Linear Regression (baseline), Random Forest Regressor (tree ensemble), XGB Data splitting / validation strategy | Hold-out: 80% train / 20% test. On training set use k-fold cross-validation (e.g., 5-fold) for Primary evaluation metric(s) and why | Primary: Root Mean Squared Error (RMSE) â interpretable in UZS. Secondary: R-sq Expected inference input | JSON or dict with: { Location, Total_Area_sqm, Rooms, Floor, Total_Flats }. Input is preprocesse Expected inference output | Single numeric predicted Price (UZS). Additionally, return a confidence estimate (e.g., prediction Main technical risks / assumptions | Dataset representativeness for current market; limited features (no year built, condition) 7. Functional Requirements -------------------------- Requirement | How the project satisfies it:--- | :--- Provide a quantitative price estimate for a property | Regression model returns numeric price prediction in UZS for given fea Reproducibility and explainability | Provide Colab notebook, pinned requirements.txt, saved preprocessing+model pipeline. U Easy to use and test | Provide demo.ipynb with a predict() helper function and example inputs; include sample scripts showi Useful for client context | Estimates are in local currency with RMSE context and per-district reliability notes to help buyers, s 8. Expected Deliverables ------------------------ Deliverable | How it will be delivered:--- | :--- Working ML solution | demo.ipynb (Google Colab) implementing end-to-end pipeline and demo inference. Trained model or ML pipeline | model.pkl (scikit-learn Pipeline or serialized artifact bundling preprocessing + model). Dataset documentation | data/README.md with dataset source URL, license (Apache 2.0), field descriptions, and preproces Evaluation results | Test-set RMSE, MAE, R´† for Linear Regression, Random Forest, and XGBoost; cross-validation summ Inference interface | predict() function in demo.ipynb; optional small Gradio demo if requested. Reproducible repository | Full GitHub repo with notebooks, requirements.txt, README, saved model, and submission/PROJ Limitations & recommendations | Documented section outlining limits (geography, features), fairness checks, and suggestion 9. Acceptance Criteria ---------------------- Acceptance Criterion | How the project satisfies it:--- | :--- Processes previously unseen input | Saved preprocessing pipeline + model can accept new property features and return a p Output is meaningful for the client | Predictions are presented in UZS with RMSE context and per-district reliability notes. Methodology and evaluation are explainable | Notebook documents preprocessing, modeling, tuning, and evaluation; include Reproducible | Colab notebook, pinned dependencies, and fixed random seeds are provided to reproduce experiments. Known limitations, risks, and assumptions documented | Brief and data/README.md include limitations, representativeness Implementation runs per documentation | README contains step-by-step instructions to reproduce training and inference lo 10. Constraints --------------- In Scope (What this project includes) - Build and evaluate regression models (Linear Regression, Random Forest, XGBoost) on the Kaggle "House Prices in Tash - Provide a reproducible Colab notebook and serialized model/pipeline for inference. - Perform basic fairness and per-district error analysis to identify significant bias or blind spots. Out of Scope (What this project does NOT include) - Full production deployment (no hosted web or mobile application). - Real-time integration with live listing APIs. - Detailed appraisal-level features (e.g., interior condition, exact address-level micro-pricing) unless additional data is provide 11. Questions You Must Resolve ------------------------------ 1. Which features are most important for predicting house prices? - Answer approach: compute feature importance (Random Forest / XGBoost) and SHAP values; surface top 5 drivers. 2. How well does the model generalize across different districts in Tashkent? - Answer approach: per-district RMSE/MAE and bias analysis; produce maps or tables showing district-level errors. 3. How many and which features are sufficient? - Answer approach: incremental feature selection and model comparisons; consider recursive feature elimination or L1-regu 4. What is an acceptable prediction error for practical use? - Answer approach: contextualize RMSE relative to median/mean price and stakeholder expectations; propose thresholds fo 5. How can predictions be explained to a non-technical user? - Answer approach: present the top 3 contributing features for a specific prediction, show expected error range, and include 6. How to detect and handle out-of-distribution inputs? - Answer approach: implement input validation and range checks; warn when inputs fall outside training ranges and avoid ov --- Appendix â Quick Implementation Notes ------------------------------------- - Preprocessing pipeline: - Clean and normalize district names, handle missing values, validate floor â⁄ total_floors. - Numeric scaling for Linear Regression (StandardScaler); tree models not required to scale. - Encode Location: One-Hot Encoding for interpretable baselines; consider target (mean) encoding with nested CV to avoid - Use sklearn Pipeline to bundle preprocessing + model for safe serialization. - Baselines & models: - Baseline: Linear Regression (OLS) and Ridge variant for stability. - Tree-based: RandomForestRegressor (sklearn) and XGBoost XGBRegressor. - Hyperparameter search: RandomizedSearchCV or GridSearchCV with 5-fold CV on training set. - Explainability: - Feature importance (tree impurity and permutation importance). - SHAP values for local explanations (XGBoost/RandomForest). - Evaluation reporting: - Report RMSE, MAE, R´† on hold-out test set. - Provide cross-validation means and stds. - Present per-district error table and visualization (boxplots or bar charts). - Reproducibility: - Pinned dependencies in requirements.txt. - Fixed random seeds for model training and CV folds. - Step-by-step README with instructions to run demo.ipynb in Colab. If you want, I can now: - (A) Update README.md to link to this brief and demo notebook, or - (B) Produce a one-page PDF or slide-ready summary of this brief, or - (C) Insert final numeric results (test RMSE, R´†) after you run experiments and provide the numbers.